

A Comparison of Symbolic and Neuro-Symbolic Strategies for LTLf Runtime Monitoring

Author name

Date

Abstract

TODO

1 Introduction

Linear Temporal Logic (LTL) [?] is a logic formalism for expressing temporal properties of reactive systems. It extends traditional propositional logic with modal operators, allowing the specification of rules that must hold *through time*, and it is interpreted over traces of discrete events. Variants of LTL interpreted over finite traces have been proposed for modeling finite, but length-unbounded, process executions (see Sec. ??), making it suitable for finite-horizon problems that often arise in the context of AI and BPM. Temporal logics have provided one of the most popular tool in Formal Methods for verification of hardware or software system behaviors. In particular, model checking is the problem of verifying whether a model of a system meets a given specification. Despite the importance and the progress made in model checking, it is not always possible to check the model against the specification. This is due to the inherent complexity of the system or the fact that the model is only partially correct or not known. In such cases, one can resort to simpler verification approaches, such as runtime monitoring, where the system’s execution is continuously observed on-the-fly and any permanent violation or fulfillment of the specified property is immediately detected.

Standard symbolic techniques for both model checking and runtime monitoring have achieved great success, but they also suffer for some inherent limitation. Among the others there is the difficulty in dealing with the increasing growth of big data and the lack of flexibility in dealing with highly unpredictable environments, say e.g. environments with noisy sensors or probabilistic uncertainties, and adapting the specification when soft violations could happen. Conversely, pure deep learning architectures, such as Recurrent Neural Networks (RNNs) and Transformers, excel at modeling high-dimensional, noisy data distributions, but they likely generate outputs that violate critical safety constraints, as they lack a mechanism to adhere to formal logics. This naturally leads to the use of hybrid approaches, which can possibly overcome some of the mentioned limitations.

In this paper, we will compare different techniques for runtime monitoring, from purely symbolic methods to the state-of-the-art neuro-symbolic architectures. We start revisiting the neural-symbolic approach to runtime monitoring, originally proposed by [?, ?] through the RuleRunner system [?]. The authors claimed that the resulting neural monitors could exploit GPU parallelism for efficient verification, outperforming purely symbolic mechanisms. Since then, some significant developments have reshaped the landscape. First, LTL_f [?] has become the standard formalism

for temporal reasoning over finite traces in AI, bringing with it mature tools for compiling formulas into DFAs. Then, differentiable automata representations such as DeepDFA [?] have emerged, enabling the integration of temporal logic knowledge into neural architectures through gradient-based learning. **Possibly include other NeSy approaches.** We present a systematic comparison of three paradigms for LTL_f runtime monitoring: (i) purely symbolic DFA-based monitoring, (ii) rule-based neural-symbolic monitoring in the RuleRunner tradition, and (iii) automata-based neural-symbolic monitoring using DeepDFA.

TODO: Mention briefly the experimental results obtained.

2 Background

2.1 An Historical Overview of Temporal Logics over Finite Traces

Studying LTL over finite traces gained significant attention [?, ?]. The original RuleRunner system [?] refers to FLTL as a finite-trace variant of LTL. As already mentioned, in the last decade LTL_f has established as the standard language for modeling finite traces of executions. Notably, FLTL and LTL_f share essentially the same syntax and semantics, offering two next operators, with a strong next operator X which is false at the last position if no successor exists, and a weak next operator W which is true at the last position (see Sec. ?? for details). FLTL makes explicit use of an *END* symbol for signaling the last element of the trace, but this can be easily expressed with the weak next operator by $END \equiv W\text{false}$. In this paper, we use LTL_f as the specification language for all three monitoring paradigms we compare.

TODO: Mention other finite-trace variants.

2.2 Syntax and Semantics

Given a finite set Σ of atomic propositions, an LTL_f formula φ has the form:

$$\varphi ::= \top \mid \perp \mid a \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid X\varphi \mid \varphi_1 U \varphi_2,$$

where $a \in \Sigma$. We use $\top$ and $\perp$ to denote *True* and *False*, respectively. X (Strong Next) and U (Until) are temporal operators. Other temporal operators can be derived as follows: W (Weak Next) is defined as $W\varphi \equiv \neg X\neg\varphi$, R (Release) is defined as $\varphi_1 R \varphi_2 \equiv \neg(\neg\varphi_1 U \neg\varphi_2)$, G (globally) is defined as $G\varphi \equiv \perp R \varphi$, and F (eventually) is defined as $F\varphi \equiv \top U \varphi$.

A trace $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_T)$ is a sequence of propositional assignments to the propositions in Σ , where $\sigma_t \in 2^\Sigma$ is the set of all and only propositions that are true at instant t . Additionally, $|\sigma| = T$ denotes the length of the trace. Note that, if the propositional symbols in Σ are all *mutually exclusive*, i.e., the domain produces exactly one symbol true at each step, then we have $\sigma_t \in \Sigma$. Let $last = |\sigma| - 1$. We inductively define when an LTL_f formula φ is true at an instant i (for $0 \leq i \leq last$), written $\sigma, i \models \varphi$, as follows:

- $\sigma, i \models a$ iff $a \in \sigma_i$, for $a \in \Sigma$;
- $\sigma, i \models \neg\varphi$ iff $\sigma, i \not\models \varphi$;
- $\sigma, i \models \varphi_1 \wedge \varphi_2$ iff $\sigma, i \models \varphi_1$ and $\sigma, i \models \varphi_2$;
- $\sigma, i \models X\varphi$ iff $i < last$ and $\sigma, i + 1 \models \varphi$;

- $\sigma, i \models \varphi_1 \mathcal{U} \varphi_2$ iff for some j such that $i \leq j \leq \text{last}$, we have $\sigma, j \models \varphi_2$, and for all k such that $i \leq k < j$, we have $\sigma, k \models \varphi_1$.

We say that σ satisfies ϕ , written $\sigma \models \varphi$, if $\sigma, 0 \models \varphi$. A formula φ is satisfiable if it is true wrt some trace σ , and is valid, if it is true wrt every trace σ .

2.3 Symbolic Monitoring

Deterministic Finite Automata. Any LTL_f formula φ can be translated into a Deterministic Finite Automaton (DFA) [?] $A_\varphi = (2^\Sigma, Q, q_0, \delta, F)$, where 2^Σ is the automaton alphabet, Q is the finite set of states, $q_0 \in Q$ is the initial state, $\delta : Q \times 2^\Sigma \rightarrow Q$ is the transition function, and $F \subseteq Q$ is the set of final states. We also define the extended transition function over traces $\delta^* : Q \times (2^\Sigma)^* \rightarrow Q$ recursively as:

$$\begin{aligned} \delta^*(q, \epsilon) &= q \\ \delta^*(q, \sigma + \mathbf{x}) &= \delta^*(\delta(q, \sigma), \mathbf{x}), \end{aligned} \tag{1}$$

where $\sigma \in 2^\Sigma$ is a symbol and $\mathbf{x} \in (2^\Sigma)^*$ is a trace. The automaton accepts the trace σ iff $\delta^*(q_0, \sigma) \in F$, and in that case we say that σ belongs to the language of the automaton, denoted as $\mathcal{L}(A_\varphi)$. We have that φ and A_φ are equivalent because, for any trace $\sigma \in (2^\Sigma)^*$:

$$\sigma \in \mathcal{L}(A_\varphi) \iff \sigma \models \varphi. \tag{2}$$

We can rely on the use of DFAs for several reasoning tasks, like model checking, planning and synthesis. In particular, monitoring formula φ becomes a deterministic walk over the automaton A_φ : the monitor keeps a single current state q , initialized to q_0 , and on observing the time point $\sigma_t \in 2^\Sigma$ it advances $q \leftarrow \delta(q, \sigma_t)$. Because the automaton is fixed and deterministic, each step costs a single transition lookup, independent of the trace seen so far and of the size of φ . In the following, DFA-based monitoring will serve as the exact baseline against which the neuro-symbolic paradigms will be measured.

Verdicts and early termination. Online monitoring requires a three-valued runtime semantics [?]: at each step the monitor emits SATISFY, VIOLATE, or UNDECIDED, the last meaning that the observed prefix can still be extended into both an accepting and a non-accepting continuation. A definite verdict can be issued as soon as the outcome is settled regardless of the future, which corresponds to two structural properties of the current state, both precomputed once by backward reachability over A_φ . A state is a *trap* if no accepting state is reachable from it: reaching a trap means φ is permanently violated, so the monitor halts with VIOLATE. Dually, a state is an *accepting sink* if every state reachable from it (including itself) is accepting: the monitor halts with SATISFY. With trap and sink sets materialized at compile time, the per-step verdict reduces to two set-membership tests on top of the transition lookup. As long as the current state is neither, the verdict is UNDECIDED and monitoring continues. Some specifications never reach a trap or a sink, so STEP returns UNDECIDED for the entire trace. An example is the response pattern $G(a \rightarrow Fb)$, which can always be satisfied by a later b and always be rescued from violation, hence has neither an accepting sink nor a trap. For these the verdict is only binary at the end of the trace, and the monitor reports a final verdict by testing acceptance of the state reached, i.e. SATISFY iff $\delta^*(q_0, \sigma) \in F$ and VIOLATE otherwise.

3 Rule-based Neuro-Symbolic Monitoring: RuleRunner

3.1 Architecture

A RuleRunner system is a tuple $\langle S, R_E, R_R \rangle$, where S is the state of the system, R_E is the set of *evaluation rules*, and R_R is the set of *reactivation rules*. In details:

- The state S contains observations, active rules, and truth evaluations. $R[\phi]$ denotes an active rule, while $[\phi]V$ denotes a 3-values truth evaluation, with $V \in \{T, F, ?\}$.
- Evaluation rules are used to compute the truth value of a formula ϕ under verification, given the truth values of its direct subformulae ψ_i . They have the form $R[\phi], [\psi_1]V, \dots, [\psi_n]V \rightarrow [\phi]V$. For instance, $R[a \vee b], [a]T \rightarrow [a \vee b]T$, because the disjunction is evaluated to *true* if one of the two disjuncts is true.
- Reactivation rules are used when a formula ϕ is evaluated to undecided, i.e. $[\phi]?$, in which case that formula and its subformulae are scheduled to be monitored again at the next time point of the trace. They have the form $[\phi]? \rightarrow R[\phi], R[\psi_1], \dots, R[\psi_n]$. For instance, $[\Diamond b]? \rightarrow R[b], R[\Diamond b]$, because if $\Diamond b$ is undecided, we need to re-evaluate b and $\Diamond b$ at the next step.

Both active rules and truth evaluations can be enriched by a *qualifier*, such as B (binary), L (left) and R (right), which provides additional information recording which operands within a formula/subformula are still being watched. For instance, $R[a \wedge \Diamond b], [a]T \rightarrow [a \wedge \Diamond b]?R$, because if the left conjunct a is true, it is still needed to check the truth value right conjunct $\Diamond b$ to assign a truth value to the conjunction.

Verification of a trace against a formula proceeds iteratively. At the beginning of each iteration (corresponding to the monitoring of a time point), the state S contains a set of rule names corresponding to the subformulae to be checked at that time point. First, the procedure includes observations of the current time point in the state S and fires the evaluation rules repeatedly incrementing the state itself until it reaches a fixpoint. In practice, this means computing the truth values of all subformulae of ϕ in a bottom-up way, from simple atoms to ϕ itself (see Figure ...). If the root evaluates to true (resp. false), the monitor halts with verdict SATISFY (resp. VIOLATE). Otherwise, the reactivation rules are fired, and the result constitute the state of the next iteration. Note that in this case the state is not expanded, but replaced by the result of reactivation rules. This is used to flush all the previous truth evaluation, which are to be computed from scratch at the new time point. The procedure terminates with a decided verdict because of special END-triggered evaluation rules, that returns either true or false if we consumed the whole trace. A RuleRunner system can be translated in a logic program, which in turn can be encoded in a Recurrent Neural Network with a single hidden layer, s.t. each evaluation and reactivation rule is a mapping from the input layer to the output one. For a detailed description of RuleRunner and verification algorithm see [?, ?].

The rule system underlies two behaviorally-equivalent neural encodings, both of which we evaluate in Section ???. In the *flat* encoding [?], the whole rule set is compiled into a single recurrent network following the standard CILP translation [?]: one hidden unit per rule, weights realizing each Horn clause, and the fixpoint of the evaluation phase recovered by iterating the forward pass until the fixpoint is met. In the *structured* encoding [?], the same rules are instead laid out as one CILP subnetwork per parse-tree node, wired according to the syntax of φ . This modular form is the substrate for *local learning*: because parameters are attached to syntactically-local subnetworks,

adaptation can be confined to the subtree responsible for a misclassification rather than diffused across a monolithic network.

3.2 The Nested-Temporal Limitation

RuleRunner's defining design choice is to maintain *a single shared state, with exactly one truth register per syntactic subformula*. This is what gives it a constant, compile-time rule count, a fixed-size network, and a non-branching state, and it is well suited to many flat temporal specifications. The problem is that the architecture *assumes each subformula has exactly one live truth value per time point*. This assumption is violated precisely when a temporal operator is nested inside the operand of another operator that may reinstall that operand across successive time points: the outer operator spawns a fresh instance of the operand while a prior instance of the *same* subformula is still resolving, and the two distinct live obligations are conflated onto the one shared register.

Consider the two formulas $\varphi_g = F(a \wedge b)$ and $\varphi_b = F(a \wedge Xb)$, which differ only by a single next operator, monitored over the same trace $\sigma = (\{a\}, \emptyset, \{b\})$. Neither formula is satisfied: φ_g would require a time point where a and b hold *together* (but a occurs only at $t = 0$ and b only at $t = 2$), and φ_b would require a time point t with $a \in \sigma_t$ and $b \in \sigma_{t+1}$ (but the only a is at $t = 0$, and is not followed by a b at $t = 1$). The correct verdict in both cases is therefore VIOLATE, but RuleRunner can provide the correct answer only for the first formula.

First, consider φ_g . The run is:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$[a]T, [b]F \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$	$\rightarrow R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
1	$\emptyset$	$[a]F, [b]F \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$	$\rightarrow R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
2	$\{b\}$	$[a]F, [b]T \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$ $[F(a \wedge b)]? \rightarrow_{\text{END}} [F(a \wedge b)]F$	$\rightarrow \text{VIOLATE}$

Now, consider φ_b . In this case, the run diverges:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$R[Xb] \rightarrow [Xb]?$ $[a]T, [Xb]? \rightarrow [a \wedge Xb]?R$ $[a \wedge Xb]?R \rightarrow [F(a \wedge Xb)]?$	$\rightarrow R[Xb]M, R[b]$ $\rightarrow R[a \wedge Xb]R$ $\rightarrow R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
1	$\emptyset$	$R[Xb] \rightarrow [Xb]?$ $R[Xb]M, [b]F \rightarrow [Xb]F$ $R[a \wedge Xb]R, [Xb]? \rightarrow [a \wedge Xb]?R$ $R[a \wedge Xb]R, [Xb]T \rightarrow [a \wedge Xb]T$ $R[a \wedge Xb], [a]F \rightarrow [a \wedge Xb]F$ $[a \wedge Xb]?R \rightarrow [F(a \wedge Xb)]?$	$\rightarrow R[Xb]M, R[b]$ $\rightarrow R[a \wedge Xb]R$ $\rightarrow R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
2	$\{b\}$	$R[Xb] \rightarrow [Xb]?$ $R[Xb]M, [b]T \rightarrow [Xb]T$ $R[a \wedge Xb]R, [Xb]T \rightarrow [a \wedge Xb]T$ $R[a \wedge Xb], [a]F \rightarrow [a \wedge Xb]F$ $[a \wedge Xb]T \rightarrow [F(a \wedge Xb)]T$	$\rightarrow \text{SATISFY (wrong)}$

The divergence stems from RuleRunner’s single shared state, which keeps exactly one truth register per syntactic subformula. The decisive step is the F reactivation. At every time point, an F that remains undecided reinstalls a fresh $R[Xb]$ that will defer again, while the previous time point’s deferral is still resolving via $R[Xb]M$. As mentioned previously, the state is a single flat set of truth evaluations and rule names with no position index. A reinstalling operator can place, into that one set, a freshly-deferred instance of a subformula while a prior instance of the same subformula is still resolving. The two instances are indistinguishable, and the next evaluation pass conflates them, possibly holding contradictory values for one subformula at once. In other words, the reactivation phase does not preserve the single-instance-per-subformula invariant across the whole formula.

This is a structural consequence of the one-literal-per-subformula encoding, and does not depend on the encoding of RuleRunner or on the implementation artifacts. The conflation cannot be repaired at the output layer. Consider φ_b again, the two traces $\sigma_A = (\{a\}, \emptyset, \{b\})$ (ground truth VIOLATE) and $\sigma_B = (\emptyset, \{a\}, \{b\})$ (ground truth SATISFY) for φ_b . After the second time point both present the *identical* conflicted register $[Xb] = \{F, ?\}$, yet the correct resolutions are opposite: σ_A needs the F to win, σ_B needs the $?$ to win. Any monitor whose decision is a function of the shared register alone must answer identically on both, so no priority order over the register can be correct on both traces.

3.3 A Progression-Based Reformulation of RuleRunner

The counterexample of Section ?? should not be read as a failure of the RuleRunner idea as a whole. It isolates a more precise problem: the original construction carries information across time using registers addressed only by the syntactic subformulae of the input formula. The local evaluation rules are not the source of the error — given the truth values of the direct subformulae at one time point, the extended truth tables compute the intended three-valued result. What fails is the cross-step reactivation phase: when the same subformula is reinstalled from different temporal contexts, its distinct residual meanings are written into the same register.

The repair we propose keeps the two ingredients that make RuleRunner distinctive: the two-phase evaluation/reactivation cycle, and the possibility of translating the resulting rule system into a recurrent CILP-style network. It leaves the evaluation rules R_E untouched and replaces the reactivation rules R_R with a carry addressed by *residual meaning*, not by subformula name. Concretely, instead of carrying truth registers for subformulae of the original input formula, the repaired monitor carries residual formulae obtained by *formula progression* [?]. The monitor still uses formula-indexed registers, but the relevant formulae are now drawn from the residual closure of φ , not only from its original parse tree. We call the result a sound and complete monitor: it issues SATISFY (resp. VIOLATE) at a prefix only when every finite continuation satisfies (resp. violates) φ , and at end-of-trace SATISFY iff $\sigma \models \varphi$.

Progression as the reactivation mechanism. Formula progression gives a principled account of “what remains to be checked after reading one observation”. Given a formula χ and a cell $s \in 2^\Sigma$, let $\text{prog}(\chi, s)$ denote the formula that must hold on the remaining suffix once s has been consumed. For *non-final* cells it is characterized by

$$s \cdot \tau \models \chi \iff \tau \models \text{prog}(\chi, s),$$

for every non-empty finite continuation τ . The operator is defined recursively and followed by Boolean simplification:

$$\begin{aligned}
\text{prog}(a, s) &= \top \text{ if } a \in s, \text{ and } \perp \text{ otherwise,} \\
\text{prog}(\neg\chi, s) &= \neg \text{prog}(\chi, s), \\
\text{prog}(\chi_1 \wedge \chi_2, s) &= \text{prog}(\chi_1, s) \wedge \text{prog}(\chi_2, s), \\
\text{prog}(\chi_1 \vee \chi_2, s) &= \text{prog}(\chi_1, s) \vee \text{prog}(\chi_2, s), \\
\text{prog}(X\chi, s) &= \chi, \\
\text{prog}(\chi_1 U \chi_2, s) &= \text{prog}(\chi_2, s) \vee (\text{prog}(\chi_1, s) \wedge (\chi_1 U \chi_2)),
\end{aligned}$$

with derived operators following from their definitions; in particular $\text{prog}(F\chi, s) = \text{prog}(\chi, s) \vee F\chi$ and $\text{prog}(G\chi, s) = \text{prog}(\chi, s) \wedge G\chi$. The clause for X is the non-final one: a next-obligation *falls through* to its operand, so a maturing $X\chi$ becomes the fresh, top-level obligation χ carrying no residue of the operator that spawned it — precisely the instance the shared register conflated. The finite-trace boundary is handled separately at the last cell (see below), exactly as the END-triggered rules do in the original system.

Two levels of state. The reformulation makes explicit a distinction that the original system did not need. There are now two related but different notions of state, and keeping them apart is what resolves the apparent puzzle of “a single formula progresses to a single formula, so when does the state ever become a set?”.

The first is the *semantic residual state*. At time t the monitor carries a finite *set* S_t of active residual formulae, read *conjunctively*: it denotes the residual

$$\rho_t = \bigwedge_{\chi \in S_t} \chi,$$

with the empty conjunction denoting $\top$. Semantically the state is therefore always a *single* residual ρ_t ; the set S_t is a *factored representation* of it. The monitor is initialized to

$$S_0 = \text{split}_\wedge(\text{nf}(\varphi)),$$

where nf is a semantics-preserving Boolean normalization and $\text{split}_\wedge$ breaks top-level conjunctions into separate elements, $\text{split}_\wedge(\chi_1 \wedge \chi_2) = \text{split}_\wedge(\chi_1) \cup \text{split}_\wedge(\chi_2)$ and $\text{split}_\wedge(\top) = \emptyset$, while every formula that is not a top-level conjunction is kept as a single element.

The second is the *operational RuleRunner state*. To evaluate and progress the active residuals, the monitor activates their syntactic support

$$\Gamma_t = \bigcup_{\chi \in S_t} \text{sub}(\chi),$$

and installs ordinary RuleRunner-style support registers $R[\zeta]$ and truth registers $[\zeta]V$ for $\zeta \in \Gamma_t$, together with active-root registers $A[\chi]$ for $\chi \in S_t$. The support registers are *scratch*: they are recomputed bottom-up at each time point and flushed at the next, exactly as truth evaluations are flushed in the original system. Only the active residual roots $A[\chi]$ are carried across time. This two-level view answers directly the concern that “if the state contains only the whole formula, evaluation has no intermediate parse-tree nodes to fire on”: the intermediate nodes live in Γ_t , not

in S_t . At $t = 0$, even though $S_0 = \{\varphi\}$, the support is $\Gamma_0 = \text{sub}(\varphi)$ — the *entire* parse tree of φ — so the bottom-up evaluation rules have exactly the tree they need. What changed relative to the original system is only *what is carried* (the residual roots) versus *what is regenerated* (their support).

When the state becomes a set, and why conjunction is split but disjunction is not.

Progression returns one formula, but its Boolean normal form may *be* a conjunction, which $\text{split}_\wedge$ then breaks into several roots. So the state grows into a genuine multi-element set exactly when progression produces a top-level conjunction — which is what G , the left recursion of U , and any top-level $\wedge$ in φ do, but F does not. This is why the singleton example $\varphi_b = F(a \wedge Xb)$ is misleading: its residual $b \vee F(a \wedge Xb)$ is a top-level *disjunction* and stays one element, whereas $\varphi = G(a \rightarrow Fb)$ oscillates between the states $\{G(a \rightarrow Fb)\}$ and $\{Fb, G(a \rightarrow Fb)\}$, the second arising the moment an unfulfilled a opens a pending eventuality Fb alongside the invariant.

Splitting conjunctions is not merely cosmetic, but neither is it semantically load-bearing: since $\{\chi_1, \chi_2\}$ and $\{\chi_1 \wedge \chi_2\}$ denote the same residual, one *could* carry ρ_t as a single monolithic formula and never split. Two reasons make the factored form the right choice. First, *soundness is free for $\wedge$* : progression is a homomorphism for conjunction, $\text{prog}(\chi_1 \wedge \chi_2, s) = \text{prog}(\chi_1, s) \wedge \text{prog}(\chi_2, s)$, so progressing the conjuncts element-wise progresses the residual they denote. Second, it *keeps the monitor RuleRunner-shaped*: each conjunct is its own register (its own CILP subnetwork), so on the flat fragment RuleRunner already handled correctly the state stays exactly as compact and local as before. The decisive asymmetry is that disjunction is *never* split. A disjunctive residual such as $b \vee F(a \wedge Xb)$ records a *correlation* — “either the b owed by the just-seen a arrives next, or the eventuality is still open” — that is destroyed the instant the two sides are carried independently. Storing each disjunctive obligation as one atomic element is exactly what the per-subformula valuation could not do, and what completeness requires.

The revised two-phase cycle. Each step keeps the original two-phase structure, now over the residual closure rather than the fixed parse tree of φ .

1. **Evaluation.** The current observation σ_t is injected, and the unchanged rules R_E fire bottom-up over the support Γ_t to a fixpoint. This is a cheap three-valued read-out: if every active root reduces to $\top$ then $\rho_t \equiv \top$ and the monitor halts with SATISFY; if any reduces to $\perp$ it halts with VIOLATE; otherwise the verdict is UNDECIDED. Crucially, this collapse can already fire mid-trace — a disjunct going $\top$ (or a conjunct going $\perp$) decides the root without waiting for the end — which is why an evaluation phase is retained rather than reading the verdict off the reactivation output alone.
2. **Reactivation by progression.** If undecided, each active root is progressed independently through the rule schema

$$R[\chi], \sigma_t \longrightarrow R[\text{prog}(\chi, \sigma_t)] \quad (\chi \in S_t),$$

and the results are re-collected into the next residual and re-split:

$$\rho_{t+1} = \text{nf} \left(\bigwedge_{\chi \in S_t} \text{prog}(\chi, \sigma_t) \right), \quad S_{t+1} = \text{split}_\wedge(\rho_{t+1}),$$

discarding any $\top$ conjunct. This supersedes the operator-specific reinstall rules $[\psi]? \rightarrow R[\psi], R[\psi_1], \dots, R[\psi_n]$ of the original system.

It is essential that reactivation progresses only the active *roots* S_t , not every undecided subformula in Γ_t . Reactivating all undecided subformulae would reintroduce the original defect: if the active residual is $b \vee F(a \wedge Xb)$ and b holds now, the whole residual is already satisfied, and independently carrying $F(a \wedge Xb)$ forward merely because it is a subformula of the disjunction would resurrect the very disjunctive correlation the fix is meant to preserve. The support subformulae in Γ_t are scratch used to *compute* the progression of the roots; they carry no temporal meaning of their own.

The two phases remain genuinely distinct — a bottom-up verdict read-out and a carry update, both sets of Horn clauses — so the reformulated system still compiles to a single-hidden-layer recurrent network, and remains differentiable under the same $\text{sign} \rightarrow \tanh$ relaxation used for the adaptation setting of Section ???. Conjunctive obligations still factor, so on the flat fragment the state is as compact as before; what does *not* factor is disjunction, so the number of *distinct* residual roots reachable from φ is what governs the state size, and in the worst case it matches the size of the minimal DFA of φ (see Section ???).

Finiteness and two realizations. Let $\mathcal{R}_\varphi$ be the set of all formulae that can occur as active residual roots by iterating this construction from φ , and let $\mathcal{C}_\varphi = \bigcup_{\chi \in \mathcal{R}_\varphi} \text{sub}(\chi)$ be its subformula closure. Quotiented by logical equivalence, $\mathcal{R}_\varphi$ is finite — its elements are Boolean combinations over the closure of φ — which yields two realizations. Run *lazily*, the monitor carries the residual set and progresses-and-simplifies it on the fly, materializing only the roots actually visited on a given trace; it never constructs the full state space and can stay far smaller than it when the visited fragment is small. Run *eagerly*, one precomputes the reachable $\mathcal{R}_\varphi$, $\mathcal{C}_\varphi$, and the induced transition “residual-set $\times$ observation $\rightarrow$ residual-set” once, obtaining a fixed, finite rule set that is again CILP-encodable and differentiable. The two are the rule-side analogue of on-the-fly versus explicit-state automaton construction; indeed, materialized eagerly and quotiented, the transition system *is* an automaton equivalent to φ , progression being the standard route from LTL_f to its (alternating, then determinized) automaton $[\cdot, \cdot]$, here assembled compositionally one operator rule at a time rather than by monolithic determinization. We use this correspondence only to place the construction; the point of keeping the rule form is not a faster monitor but the structured, formula-indexed, differentiable state it carries — its units are attached to residual formulae and their syntactic components, rather than to anonymous DFA states.

The counterexample, revisited. Progression resolves the divergence of Section ???. Recall $\varphi_b = F(a \wedge Xb)$, write $\psi = a \wedge Xb$, and take the two traces $A = (\{a\}, \emptyset, \{b\})$ (ground truth VIOLATE) and $B = (\emptyset, \{a\}, \{b\})$ (ground truth SATISFY) that the one-register state cannot tell apart. Since $\text{prog}(F\psi, s) = \text{prog}(\psi, s) \vee F\psi$, reading a non-final cell in which a holds produces the residual $b \vee F\psi$, carried as a *single* atomic root. The active residual roots before each cell are:

	before σ_0	before σ_1	before σ_2	last-cell verdict
A	$\{F\psi\}$	$\{b \vee F\psi\}$	$\{F\psi\}$	VIOLATE
B	$\{F\psi\}$	$\{F\psi\}$	$\{b \vee F\psi\}$	SATISFY

On A the disjunctive obligation opened by the a at time 0 appears before σ_1 ; at σ_1 no b holds, the immediate disjunct disappears, and the residual falls back to $F\psi$. The last cell brings $\{b\}$ but

no pending residual linking it to the earlier a , and $F\psi$ is false on the one-cell suffix: VIOLATE. On B the same obligation is opened one time point later, so it is still live as the disjunct b when $\{b\}$ arrives, the residual reduces to $\top$: SATISFY. The decisive state is the single disjunctive root $b \vee F(a \wedge Xb)$ — one atomic element of $\mathcal{R}_{\varphi_b}$ — which occurs at *different* time points on the two traces and which the one-register valuation cannot represent at all.

Correctness. We now state the correctness property. Let $\text{last}(\chi, s) \in \{\top, \perp\}$ denote the truth value of χ on the one-cell trace (s) : atoms and Boolean connectives are evaluated as usual, and the temporal boundary cases are

$$\text{last}(X\chi, s) = \perp, \quad \text{last}(W\chi, s) = \top, \quad \text{last}(\chi_1 U \chi_2, s) = \text{last}(\chi_2, s),$$

the finite-trace counterpart of the END-triggered rules.

Theorem 1 (Correctness of progression-based RuleRunner). *Let φ be an LTL_f formula and let RR_φ^{prog} be the eager progression-based RuleRunner monitor constructed as above. Then, for every finite non-empty trace σ , the monitor returns SATISFY iff $\sigma \models \varphi$, and VIOLATE iff $\sigma \not\models \varphi$. Moreover, any verdict returned before the last cell is sound for all finite continuations of the prefix read so far.*

Proof. Let $\sigma = (\sigma_0, \dots, \sigma_{T-1})$ be a finite non-empty trace. For $0 \leq t < T$, let S_t be the active residual state before reading cell σ_t , i.e. after the prefix $\sigma_0 \dots \sigma_{t-1}$, and let $\rho_t = \bigwedge_{\chi \in S_t} \chi$ be the formula it denotes. By construction $S_0 = \text{split}_\wedge(\text{nf}(\varphi))$, so $\rho_0 \equiv \varphi$.

Step property of progression. For every formula χ , non-final cell s , and non-empty finite continuation τ ,

$$s \cdot \tau \models \chi \iff \tau \models \text{prog}(\chi, s), \quad (3)$$

by structural induction on χ . The atomic and Boolean cases are immediate. For $X\chi$, since τ is non-empty, $s \cdot \tau \models X\chi$ iff $\tau \models \chi$, matching $\text{prog}(X\chi, s) = \chi$; the weak-next case is identical on non-final cells. For $\chi_1 U \chi_2$, $s \cdot \tau \models \chi_1 U \chi_2$ iff χ_2 holds at the current cell or χ_1 holds there and $\chi_1 U \chi_2$ holds on the suffix; by the induction hypothesis this is $\tau \models \text{prog}(\chi_2, s) \vee (\text{prog}(\chi_1, s) \wedge (\chi_1 U \chi_2))$. Derived operators inherit the property from their definitions.

Residual invariant. We show, for every $0 \leq t < T$ and non-empty finite continuation τ ,

$$\sigma_0 \dots \sigma_{t-1} \cdot \tau \models \varphi \iff \tau \models \rho_t. \quad (4)$$

For $t = 0$ this holds because $\rho_0 \equiv \varphi$. Assume it holds for some $t < T - 1$. By the reactivation rule and the fact that progression distributes over conjunction up to logical equivalence,

$$\rho_{t+1} \equiv \text{nf} \left(\bigwedge_{\chi \in S_t} \text{prog}(\chi, \sigma_t) \right) \equiv \text{nf}(\text{prog}(\rho_t, \sigma_t)).$$

Hence, for every non-empty continuation τ ,

$$\begin{aligned} \sigma_0 \dots \sigma_t \cdot \tau \models \varphi &\iff \sigma_t \cdot \tau \models \rho_t \quad (\text{induction hypothesis}), \\ &\iff \tau \models \text{prog}(\rho_t, \sigma_t) \quad (\text{Eq. (??)}), \\ &\iff \tau \models \rho_{t+1} \quad (\text{nf preserves semantics}). \end{aligned}$$

Last cell. Applying Eq. (??) with $t = T-1$ and continuation (σ_{T-1}) gives $\sigma \models \varphi \iff (\sigma_{T-1}) \models \rho_{T-1}$, i.e. $\text{last}(\rho_{T-1}, \sigma_{T-1}) = \top$, which is exactly what the final evaluation rules compute. Hence the monitor returns SATISFY exactly on traces satisfying φ , and VIOLATE otherwise.

Operational realization and early termination. The support $\Gamma_t = \bigcup_{\chi \in S_t} \text{sub}(\chi)$ contains every subformula needed to evaluate and progress the active roots, and the eager monitor allocates registers for all formulae in $\mathcal{C}_\varphi$, so the bottom-up computation is realized by finitely many rules; since only roots in S_t are reactivated, support subformulae acquire no independent temporal meaning, and the rule-based monitor implements exactly the residual update above. Finally, if an early verdict is returned, ρ_t is valid resp. unsatisfiable over all finite continuations; by Eq. (??) every completion of the prefix read so far then has the same truth value for φ , so early termination is sound. These are precisely the accepting-sink resp. trap conditions of Section ??, precomputed once in the eager realization exactly as the sink and trap sets are; the lazy realization detects them by Boolean simplification, a sound under-approximation that may lag by a few time points but never issues a wrong verdict. $\square$

TODO: in the extended version, spell out the per-operator progression-correctness lemma and the end-of-trace resolution in full, and state the $\mathcal{R}_\varphi$ -vs-minimal-DFA size bound as a separate proposition.

4 Automata-based Neuro-Symbolic Monitoring: DeepDFA

4.1 Architecture

DeepDFA [?] keeps the automaton of the symbolic paradigm but replaces its explicit state walk with a differentiable, tensorized one; we adopt the same forward formulation used for predictive monitoring of business processes by [?]. In its original formulation DeepDFA is a *probabilistic relaxation* of a DFA: a probabilistic finite automaton whose transition tensor is real-valued and *learnable*, with each transition row constrained to the probability simplex (e.g. through a softmax or a Gumbel–Softmax relaxation of discrete choices [?, ?]), and the crisp automaton recovered only in the zero-temperature, one-hot limit. Accordingly, starting from the minimal DFA $A_\varphi = (2^\Sigma, Q, q_0, \delta, F)$ of Section ??, the general object is a three-way tensor $T \in [0, 1]^{|Q| \times |2^\Sigma| \times |Q|}$ that is row-stochastic in its last index, $\sum_{q'} T[q, \sigma, q'] = 1$, together with an acceptance vector $v_o \in [0, 1]^{|Q|}$. When the automaton is instead *compiled* from a known LTL_f formula — the setting of this paper’s monitoring experiments — these parameters are fixed to their exact one-hot values, $T \in \{0, 1\}^{|Q| \times |2^\Sigma| \times |Q|}$ with $T[q, \sigma, q'] = 1$ iff $\delta(q, \sigma) = q'$, and $v_o \in \{0, 1\}^{|Q|}$ with $v_o[q] = 1$ iff $q \in F$; this crisp instantiation is what the monitor uses, while the real-valued parameterization is the substrate for the gradient-based adaptation of Section ?. In either case the monitor state is no longer a single automaton state but a row vector $\mathbf{q}_t \in [0, 1]^{|Q|}$ over states, initialized to the one-hot vector $\mathbf{q}_0 = v_i$ of q_0 . Writing $T_\sigma = T[\cdot, \sigma, \cdot] \in [0, 1]^{|Q| \times |Q|}$ for the (row-stochastic) transition matrix of symbol σ , a step is a single vector–matrix product

$$\mathbf{q}_t = \mathbf{q}_{t-1} T_{\sigma_t}, \quad \text{acc}_t = \mathbf{q}_t \cdot v_o, \quad (5)$$

where $\text{acc}_t \in [0, 1]$ is the acceptance score of the prefix read so far. On the compiled tensor every T_{σ_t} is a 0/1 deterministic-transition matrix and, on crisp observations, $\mathbf{q}_t$ stays one-hot, so the update reproduces the symbolic walk exactly: $\mathbf{q}_t$ is the indicator of $\delta^*(q_0, \sigma_1 \cdots \sigma_t)$. The three-valued

verdict and early termination are then read off exactly as in Section ??, by testing membership of $\operatorname{argmax}_q \mathbf{q}_t[q]$ in the precomputed trap and accepting-sink sets, and the final verdict tests its acceptance.

4.2 Soft Observations and Differentiability

The reason to pay for this tensorization is that Equation (??) extends verbatim to *probabilistic* observations, which the symbolic walk cannot represent. Suppose a time point is delivered not as a crisp assignment but as a vector of per-atom probabilities $\mathbf{p}_t \in [0, 1]^\Sigma$, as produced by a neural perceptor or a noisy sensor. We replace the one-hot symbol selection by an *expected* transition matrix $M(\mathbf{p}_t) \in [0, 1]^{|Q| \times |Q|}$, whose entry (q, q') is the probability that the guard of edge $q \rightarrow q'$ is satisfied under $\mathbf{p}_t$. Here our setting departs from the source formulation: the original DeepDFA and the process-monitoring work that adopts it assume *mutual exclusivity* — exactly one activity is true per step — so the alphabet is the set of atoms and the expected transition is a simple convex combination $M(\mathbf{p}_t) = \sum_\sigma p_\sigma T_\sigma$ of the one-hot per-symbol matrices, weighted by the categorical symbol probabilities. Because our propositional benchmark family admits conjunctions of simultaneously-true atoms (Section ??), we drop this assumption and instead compute each edge’s guard-satisfaction probability compositionally over the boolean structure of the guard under an atom-independence assumption ($P(a) = p_a$, $P(\neg\psi) = 1 - P(\psi)$, $P(\psi_1 \wedge \psi_2) = P(\psi_1)P(\psi_2)$, $P(\psi_1 \vee \psi_2) = 1 - (1 - P(\psi_1))(1 - P(\psi_2))$). The update and acceptance score are unchanged,

$$\mathbf{q}_t = \mathbf{q}_{t-1} M(\mathbf{p}_t), \quad \text{acc}_t = \mathbf{q}_t \cdot v_o, \quad (6)$$

so that $\mathbf{q}_t$ propagates the perceptual uncertainty all the way to the verdict rather than discarding it through a hard 0.5 threshold, and acc_t is a soft acceptance score that the symbolic walk fundamentally cannot emit. Because each $M(\mathbf{p}_t)$ is a smooth function of $\mathbf{p}_t$ (and, if desired, of the learnable parameters of T), the whole monitor is differentiable: gradients of a verdict loss flow back through the automaton, which is what makes DeepDFA the vehicle for gradient-based specification adaptation and, ultimately, end-to-end training of a perceptor. On crisp inputs $M(\mathbf{p}_t)$ collapses to T_{σ_t} and the probabilistic semantics is exact for any guard. On fractional inputs, however, the guarantee is conditional: the compositional score is exact — and $M(\mathbf{p}_t)$ genuinely row-stochastic, so acc_t is a well-defined acceptance *probability* — precisely when every guard is *read-once* (each atom occurring once), which holds for the LTL_f families we study after MONA’s factoring. On a non-read-once guard the independence assumption double-counts shared atoms across disjuncts: the per-state out-guard probabilities no longer sum to one, $M(\mathbf{p}_t)$ ceases to be row-stochastic, and the raw acc_t is no longer a valid probability (it can exceed 1). It can be renormalized by the total propagated mass $\mathbf{q}_t \cdot \mathbf{1}$ to restore the range $[0, 1]$ — a no-op on read-once guards, where the mass is already one — but this restores range validity, not calibration; we return to this point, and to which probabilistic quantity a monitor *should* report, in Section ?? and the experiments.

4.3 The Alphabet Blow-up

DeepDFA’s structural cost is dual to those of the other two paradigms. The transition tensor is indexed by the full alphabet 2^Σ , so a dense materialization is exponential in the number of atoms, $|T| = |Q|^2 \cdot 2^{|\Sigma|}$. One can avoid building the dense tensor by evaluating the per-edge guard probabilities on the fly — the factored form underlying Equations (??)–(??), which never enumerates 2^Σ — and this is what we use for specifications over many atoms. But the blow-up

is not merely an implementation choice: when a single guard depends on all $|\Sigma|$ atoms there is in general no sub-exponential set of symbols to sum over, and only the read-once circuit structure of a guard (which a flat DFA does not expose) permits its satisfaction probability to be computed in time linear in $|\Sigma|$.

Concretely, the monitor admits two representations of the transition function, and we evaluate both. The *dense* representation materializes the full tensor T and executes each step as a single (batched) matrix product indexing the observed symbol; it is the fastest option whenever the alphabet fits in memory and is the natural showcase for cross-trace batching, but it becomes infeasible past roughly a dozen atoms, where $2^{|\Sigma|}$ exhausts memory. The *factored* representation, developed next, never builds T and is what we use for specifications over many atoms.

4.4 The Factored Transition Representation

The factored representation exploits the fact that the DFA edges produced by MONA are labelled not by individual symbols but by *symbolic boolean guards* over the atoms (e.g. $a \wedge (b \vee c)$), each standing for the set of symbols that satisfy it. This is the same compact edge labelling that lets the symbolic walk sidestep the alphabet (Section ??); the factored mode carries it into the tensorized setting while preserving both exact crisp monitoring and a differentiable soft path. The construction is our own assembly of standard ingredients from Boolean-function representation and knowledge compilation — Shannon expansion, disjoint sum-of-products, and circuit-based probability evaluation [?] — rather than a component inherited from the DeepDFA literature; the idea of forming an *expected* transition operator from symbol/guard probabilities is shared with recent neurosymbolic-automata work [?, ?], which likewise targets probabilistic inputs to automata in the non-mutually-exclusive setting.

We keep two complementary views of each edge guard, corresponding to the crisp and the soft regimes.

Crisp path: disjoint cube cover. At construction time each guard is expanded, by Shannon expansion, into a set of *disjoint* (mutually exclusive) cubes — partial assignments in which some atoms are required true, some required false, and the rest are don’t-cares — stored as a pair of require-true/require-false integer masks over the atoms. Given a per-atom value vector $\mathbf{p}$, each cube evaluates to $\prod_a [1 - \text{rt}_a(1 - p_a) - \text{rf}_a p_a]$ (which is p_a for a required-true atom, $1 - p_a$ for a required-false atom, and 1 for a don’t-care), and the cubes of all edges are summed into the $|Q| \times |Q|$ transition matrix by a single vectorized reduction, at a cost proportional to the number of cubes rather than to $2^{|\Sigma|}$. On crisp 0/1 inputs each cube is 0/1 and, being mutually exclusive, the cubes of a state’s out-guards partition the assignment space and sum to a 0/1 row-stochastic transition, so this path reproduces the symbolic walk exactly. This is the path our monitoring experiments time, and it is what keeps the factored curve flat in $|\Sigma|$.

Crucially, the factored representation is *not* an unconditional escape from the alphabet blow-up: Shannon expansion of a guard on k atoms can produce up to $\Theta(2^k)$ disjoint cubes, so a guard that genuinely depends on all $|\Sigma|$ atoms with no compact orthogonal cover reinstates the exponential cost in the number of cubes. What the factored form buys is a shift of the blow-up from “always $2^{|\Sigma|}$ ” (the dense tensor) to “ $2^{|\Sigma|}$ only for adversarial guards”: a genuine and often large win on the structured, read-once guards that arise in practice (our LTL_f families among them), and no asymptotic improvement on pathological ones. This is consistent with the finding of Section ?? — only the read-once circuit structure of a guard admits a sub-exponential evaluation — now made

precise at the level of the representation.

Soft path: recursive guard probabilities. Separately, each guard is compiled into a differentiable closure that evaluates its satisfaction probability recursively over the boolean tree, using the independence rules of Equation (??); summing these per-edge closures yields the expected transition matrix $M(\mathbf{p}_t)$, differentiable in $\mathbf{p}_t$ and in the automaton parameters. This is the substrate for the soft observations and gradient-based adaptation of Section ?? . On read-once guards the two views coincide even on fractional inputs, since the orthogonal-cube sum equals the recursive read-once probability; on crisp inputs both are exact for any guard. They part ways only on non-read-once guards with fractional inputs, and instructively so: the disjoint-cube sum computes the *exact* independent-atom marginal (it sums probabilities of mutually exclusive events), whereas the recursive closure double-counts shared atoms and can overshoot. We deliberately drive the soft readout from the recursive closure rather than the exact-marginal cube sum — both because the cube count can itself blow up, as noted above, and because retaining the overshoot keeps the non-read-once non-stochasticity of Section ?? observable rather than silently corrected. Which of these quantities is the “correct” probabilistic verdict for a monitor is itself a question we take up in Section ?? .

Our scalability experiments plot the dense and factored paths side by side — the dense curve walling out at the $2^{|\Sigma|}$ memory bound while the factored curve stays flat on the read-once families — which separates the alphabet blow-up as a *memory* wall from the residual per-step *compute* cost.

5 Theoretical Comparison

Having presented the three paradigms individually, we now compare them along the dimensions that matter for a monitor: what each can represent, where each breaks down, and — for the two neuro-symbolic paradigms — what a gradient means when the specification is adapted from data.

5.1 Three Achilles Heels

Each paradigm is exact and efficient on a large class of specifications, and each has a single structural weakness that is dual to the others’, so that no one paradigm dominates.

- **Symbolic DFA — state-space blow-up.** The LTL_f -to-DFA translation is doubly exponential in the worst case, so the compiled automaton (and hence the compile-time cost and memory) can explode even when the runtime visits only a handful of states.
- **RuleRunner — nested-temporal conflation.** The one-register-per-subformula state cannot represent disjunctions of concurrently-live obligations, so it returns wrong verdicts on temporal-under-temporal nesting (Section ??); restoring completeness forces the state up to DFA size (Section ??).
- **DeepDFA — alphabet blow-up.** The transition tensor is indexed by 2^Σ , and guards depending on all atoms admit no sub-exponential symbol set to sum over unless their read-once circuit structure is exposed (Section ??).

Two clarifications keep this comparison honest. First, the state-space blow-up is not exclusive to the symbolic paradigm: DeepDFA is compiled from the *same* minimal DFA, so it inherits $|Q|$ (its tensor is $|Q|^2 \cdot 2^{|\Sigma|}$) and suffers the same doubly-exponential worst case. We list state blow-up under the symbolic monitor because it is that monitor’s *only* structural weakness, whereas for DeepDFA the *additional*, representation-specific cost that the symbolic monitor escapes is the alphabet axis. Second, the two neuro-symbolic-versus-symbolic heels are duals in a precise sense: a DFA transition function is a table with $|Q| \cdot 2^{|\Sigma|}$ entries, and each paradigm compresses a different axis of it to obtain its advantage, paying on the axis it leaves explicit. The symbolic monitor compresses the 2^Σ axis with symbolic guards — keeping an explicit current state, so its cost driver is $|Q|$ — and this is exactly why it cannot suffer an alphabet blow-up: it never enumerates symbols. DeepDFA (dense) compresses the state axis into a uniform, differentiable, batchable matrix product — which requires one explicit transition matrix per symbol, so its cost driver is $2^{|\Sigma|}$. Neither weakness can be swapped onto the other paradigm without discarding the very representational choice that defines it; the factored representation (Section ??) is precisely the attempt to compress *both* axes at once, and recovers the flat-in- $|\Sigma|$ scaling on the read-once guards where such a compression exists.

TODO: capability matrix summarizing exactness, soft-input support, differentiability, and the three heels across the three paradigms.

5.2 Repaired RuleRunner and DeepDFA as Learnable Objects

As *monitors*, the completeness-restored RuleRunner and DeepDFA are interchangeable: the progression construction converges to the same minimal DFA (Section ??), so on any crisp trace they agree, and on crisp throughput the symbolic walk is optimal for both. The reason to keep both lineages is not runtime but what they expose to gradient-based adaptation.

DeepDFA’s learnable parameters are the entries of the transition tensor and the accepting vector: a gradient adapts transition probabilities between *opaque* states, over $|Q|^2 \cdot |\Sigma|$ parameters with no structural inductive bias, and the learned object can drift into an arbitrary weighted automaton no longer corresponding to any LTL_f formula. The rule-based construction, by contrast, attaches parameters to the *rules*, indexed by subformula and operator at the symbolic construction level: adaptation is syntactically local, the learned object re-extracts as a corrected specification, and the parameterization carries an inductive bias toward valid specifications. The two are thus interchangeable as monitors but not as learnable objects — DeepDFA adapts an opaque automaton, the rule form adapts a structured specification — which is the distinction we build on in the specification-adaptation experiments (Section ??) and the bridge to follow-up work.

TODO: expand once the adaptation experiments are in; cite Bacchus–Kabanja progression and the LTL_f 2EXP bound as primary sources.

6 Experimental Comparison

All three monitoring paradigms are implemented in a common framework sharing a uniform MONITOR interface, which ensures that the experimental harness is paradigm-agnostic: timing experiments iterate over a list of monitor instances without any paradigm-specific branching. The shared compilation pipeline uses `ltlf2dfa` [?] to translate each LTL_f formula into a minimal DFA once at construction time.

We evaluate the monitoring paradigms along three axes, using synthetic traces throughout

Experiments 1–3 to isolate per-step monitoring cost from data-dependent effects such as early-termination frequency. This follows the methodology of experiments in [?].

7 Related Work

8 Conclusion and Future Work

In this paper ...

8.1 Transformers

8.2 Specification Adaptation

8.3 Process Model Repair

References

- [1] Fahiem Bacchus and Froduald Kabanza. Using temporal logics to express search control knowledge for planning. *Artificial Intelligence*, 116(1-2):123–191, 2000.
- [2] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Trans. Softw. Eng. Methodol.*, 20(4):14:1–14:64, 2011.
- [3] Giuseppe De Giacomo and Marco Favorito. Compositional approach to translate LTLf/LDLf into deterministic finite automata. In Susanne Biundo, Minh Do, Robert Goldman, Michael Katz, Qiang Yang, and Hankz Hankui Zhuo, editors, *Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling, ICAPS 2021, Guangzhou, China (virtual), August 2-13, 2021*, pages 122–130. AAAI Press, 2021.
- [4] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In Francesca Rossi, editor, *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, pages 854–860. IJCAI/AAAI, 2013.
- [5] Francesco Fuggitti. Ltlf2dfa. *Zenodo*, 2019.
- [6] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [7] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. The glory of the past. In *Workshop on Logic of Programs*, pages 196–218. Springer, 1985.
- [8] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.

- [9] Nikolaos Manginas, George Paliouras, and Luc De Raedt. Nesya: Neurosymbolic automata. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025*, pages 5950–5958. ijcai.org, 2025.
- [10] Zohar Manna and Amir Pnueli. *Temporal Verification of Reactive Systems*, volume 1. Springer-Verlag, 1995.
- [11] Axel Mezini, Elena Umili, Ivan Donadello, Fabrizio Maria Maggi, Matteo Mancanelli, and Fabio Patrizi. Neuro-symbolic predictive process monitoring. *arXiv preprint arXiv:2509.00834*, 2025.
- [12] Alan Perotti, Guido Boella, and Artur d’Avila Garcez. Runtime verification through forward chaining. In *Runtime Verification: 6th International Conference, RV 2015, Vienna, Austria, September 22-25, 2015. Proceedings*, pages 185–200. Springer, 2015.
- [13] Alan Perotti, Artur d’Avila Garcez, and Guido Boella. Neural networks for runtime verification. In *2014 International Joint Conference on Neural Networks (IJCNN)*, pages 2637–2644. IEEE, 2014.
- [14] Alan Perotti, Artur d’Avila Garcez, and Guido Boella. Neural-symbolic monitoring and adaptation. In *2015 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2015.
- [15] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pages 46–57. IEEE Computer Society, 1977.
- [16] Elena Umili and Roberto Capobianco. Deepdfa: Automata learning through neural probabilistic relaxations. In *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, pages 1051–1058, 2024.